Week 6 at a glance

We will be learning and practicing to:

- Clearly and unambiguously communicate computational ideas using appropriate formalism. Translate across levels of abstraction.
  - Translating between symbolic and English versions of statements using precise mathematical language
  - Using appropriate signpost words to improve readability of proofs, including 'arbitrary' and 'assume'
- Know, select and apply appropriate computing knowledge and problem-solving techniques. Reason about computation and systems. Use mathematical techniques to solve problems. Determine appropriate conceptual tools to apply to new situations. Know when tools do not apply and try different approaches. Critically analyze and evaluate candidate solutions.
  - Writing the converse, contrapositive, and inverse of a given conditional statement
  - Determining what evidence is required to establish that a quantified statement is true or false
  - Evaluating quantified statements about finite and infinite domains
- Apply proof strategies, including direct proofs and proofs by contradiction, and determine whether a proposed argument is valid or not.
  - Identifying the proof strategies used in a given proof
  - Identifying which proof strategies are applicable to prove a given compound proposition based on its logical structure
  - Carrying out a given proof strategy to prove a given statement
  - Carrying out a universal generalization argument to prove that a universal statement is true
  - Using proofs as knowledge discovery tools to decide whether a statement is true or false
- Work with recursively defined data and prove their properties using inductive reasoning
  - Using mathematical induction to prove mathematical identities, inequalities, and other invariants
  - Using strong induction to prove mathematical identities, inequalities, and other invariants
  - Using structural induction to prove statements about recursively defined objects, e.g. strings and trees

TODO:

Review quiz based on Week 5 class material (due Monday)

Homework 3 due Tuesday

Midquarter feedback: please let us know what's working well for you and what isn't. `https://canvas.ucsd.edu/courses/77744/quizzes`

CC BY-NC-SA 2.0 Version October 30, 2026 (1)

Pre-class reading for Week 6

At this point, we've seen the proof strategies

- A **counterexample** to prove that $\forall x P(x)$ is **false**.
- A **witness** to prove that $\exists x P(x)$ is **true**.
- **Proof of universal by exhaustion** to prove that $\forall x\, P(x)$ is true when P has a finite domain
- **Proof by universal generalization** to prove that $\forall x\, P(x)$ is true using an arbitrary element of the domain.
- To prove that $\exists x P(x)$ is **false**, write the universal statement that is logically equivalent to its negation and then prove it true using universal generalization.
- To prove that $p \wedge q$ is true, have two subgoals: subgoal (1) prove p is true; and, subgoal (2) prove q is true. To prove that $p \wedge q$ is false, it's enough to prove that p is false. To prove that $p \wedge q$ is false, it's enough to prove that q is false.
- Proof of conditional by **direct proof**
- Proof of conditional by **contrapositive proof**
- Proof of disjuction using equivalent conditional: To prove that the disjunction $p \vee q$ is true, we can rewrite it equivalently as $\neg p \to q$ and then use direct proof or contrapositive proof.
- **Proof by cases**.

Recall the definitions: The set of RNA strands S is defined (recursively) by:

Basis Step:	$\mathtt{A} \in S, \mathtt{C} \in S, \mathtt{U} \in S, \mathtt{G} \in S$
Recursive Step:	If $s \in S$ and $b \in B$, then $sb \in S$

where sb is string concatenation.

The function *rnalen* that computes the length of RNA strands in S is defined recursively by:

		$rnalen : S$	$\to \mathbb{Z}^+$
Basis Step:	If $b \in B$ then	$rnalen(b)$	$= 1$
Recursive Step:	If $s \in S$ and $b \in B$, then	$rnalen(sb)$	$= 1 + rnalen(s)$

The function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively by:

		$basecount : S \times B$	$\to \mathbb{N}$	
Basis Step:	If $b_1 \in B, b_2 \in B$	$basecount(\ (b_1, b_2)\ )$	$= \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$	
Recursive Step:	If $s \in S, b_1 \in B, b_2 \in B$	$basecount(\ (sb_1, b_2)\ )$	$= \begin{cases} 1 + basecount(\ (s, b_2)\ ) & \text{when } b_1 = b_2 \\ basecount(\ (s, b_2)\ ) & \text{when } b_1 \neq b_2 \end{cases}$	

CC BY-NC-SA 2.0 Version October 30, 2026 (2)

Which proof strategies could be used to prove each of the following statements?

Hint: first translate the statements to English and identify the main logical structure.

$\forall b \in B\ \exists s \in S\ (\ basecount(\ (s, b)\ )\ > 0\ )$

$\exists s \in S\ \forall b \in B\ (\ basecount(\ (s, b)\ )\ > 0\ )$

$\exists s \in S\ (rnalen(s) = basecount(\ (s, \mathtt{A})\ )$

$\forall s \in S\ \exists b \in B\ (\ basecount(\ (s, b)\ ) > 0\ )$

$\forall s \in S\ (rnalen(s) \geq basecount(\ (s, \mathtt{A})\ ))$

$\forall s \in S\ (\ rnalen(s) > 0\ )$

CC BY-NC-SA 2.0 Version October 30, 2026 (3)

Week 6 Monday: Structural Induction

Claim $\forall s \in S$ ($rnalen(s) > 0$)

Proof: Let s be an arbitrary RNA strand. By the recursive definition of S, either $s \in B$ or there is some strand s_0 and some base b such that $s = s_0 b$. We will show that the inequality holds for both cases.

Case: Assume $s \in B$. We need to show $rnalen(s) > 0$. By the basis step in the definition of $rnalen$,

$$rnalen(s) = 1$$

which is greater than 0, as required.

Case: Assume there is some strand s_0 and some base b such that $s = s_0 b$. We will show *(the stronger claim)* that

$$\forall u \in S \; \forall b \in B \; (\; rnalen(u) > 0 \rightarrow rnalen(ub) > 0 \;)$$

Consider an arbitrary RNA strand u and an arbitrary base b, and assume towards a direct proof, that

$$rnalen(u) > 0$$

We need to show that $rnalen(ub) > 0$.

$$rnalen(ub) = 1 + rnalen(u) > 1 + 0 = 1 > 0$$

as required.

Proof by Structural Induction To prove a universal quantification over a recursively defined set:

Basis Step: Show the statement holds for elements specified in the basis step of the definition.

Recursive Step: Show that if the statement is true for each of the elements used to construct new elements in the recursive step of the definition, the result holds for these new elements.

CC BY-NC-SA 2.0 Version October 30, 2026 (4)

Claim $\forall s \in S\,(rnalen(s) \geq basecount(\ (s, \mathtt{A})\ ))$:

Proof: We proceed by structural induction on the recursively defined set S.

Basis Case: We need to prove that the inequality holds for each element in the basis step of the recursive definition of S. Need to show

$$(\ rnalen(\mathtt{A}) \geq basecount(\ (\mathtt{A}, \mathtt{A})\ )\ ) \wedge (\ rnalen(\mathtt{C}) \geq basecount(\ (\mathtt{C}, \mathtt{A})\ )\ )$$
$$\wedge(\ rnalen(\mathtt{U}) \geq basecount(\ (\mathtt{U}, \mathtt{A})\ )\ ) \wedge (\ rnalen(\mathtt{G}) \geq basecount(\ (\mathtt{G}, \mathtt{A})\ )\ )$$

We calculate, using the definitions of *rnalen* and *basecount*:

Recursive Case: We will prove that

$$\forall u \in S\ \forall b \in B\ (\ rnalen(u) \geq basecount(\ (u, \mathtt{A})\ ) \rightarrow rnalen(ub) \geq basecount(\ (ub, \mathtt{A})\ )$$

Consider arbitrary RNA strand u and arbitrary base b. Assume, as the **induction hypothesis**, that $rnalen(u) \geq basecount(\ (u, \mathtt{A})\ )$. We need to show that $rnalen(ub) \geq basecount(\ (ub, \mathtt{A})\ )$.

Using the recursive step in the definition of the function *rnalen*:

$$rnalen(ub) = 1 + rnalen(u)$$

The recursive step in the definition of the function *basecount* has two cases. We notice that $b = \mathtt{A} \vee b \neq \mathtt{A}$ and we proceed by cases.

Case i. Assume $b = \mathtt{A}$.

Using the first case in the recursive step in the definition of the function *basecount*:

$$basecount(\ (ub, \mathtt{A})\ ) = 1 + basecount(\ (u, \mathtt{A})\ )$$

By the **induction hypothesis**, we know that $basecount(\ (u, \mathtt{A})\ ) \leq rnalen(u)$ so:

$$basecount(\ (ub, \mathtt{A})\ ) = 1 + basecount(\ (u, \mathtt{A})\ ) \leq 1 + rnalen(u) = rnalen(ub)$$

and, thus, $basecount(\ (ub, \mathtt{A})\ ) \leq rnalen(ub)$, as required.

Case ii. Assume $b \neq \mathtt{A}$.

Using the second case in the recursive step in the definition of the function *basecount*:

$$basecount(\ (ub, \mathtt{A})\ ) = basecount(\ (u, \mathtt{A})\ )$$

By the **induction hypothesis**, we know that $basecount(\ (u, \mathtt{A})\ ) \leq rnalen(u)$ so:

$$basecount(\ (ub, \mathtt{A})\ ) = basecount(\ (u, \mathtt{A})\ ) \leq rnalen(u) < 1 + rnalen(u) = rnalen(ub)$$

and, thus, $basecount(\ (ub, \mathtt{A})\ ) \leq rnalen(ub)$, as required.

CC BY-NC-SA 2.0 Version October 30, 2026 (5)

To organize our proofs, it's useful to highlight which claims are most important for our overall goals. We use some terminology to describe different roles statements can have.

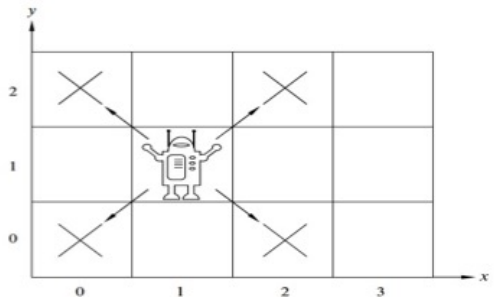
Theorem: Statement that can be shown to be true, usually an important one.

Less important theorems can be called **proposition**, **fact**, **result**, **claim**.

Lemma: A less important theorem that is useful in proving a theorem.

Corollary: A theorem that can be proved directly after another one has been proved, without needing a lot of extra work.

Invariant: A theorem that describes a property that is true about an algorithm or system no matter what inputs are used.



Theorem: A robot on an infinite 2-dimensional integer grid starts at $(0, 0)$ and at each step moves to diagonally adjacent grid point. This robot can / cannot (*circle one*) reach $(1, 0)$.

Definition The set of positions the robot can visit Pos is defined by:

Basis Step: $(0, 0) \in Pos$
Recursive Step: If $(x, y) \in Pos$, then

are also in Pos

Example elements of Pos are:

Lemma: $\forall(x, y) \in Pos$ $(x + y$ is an even integer $)$

Why are we calling this a lemma?

Proof of theorem using lemma: To show is $(1, 0) \notin Pos$. Rewriting the lemma to explicitly restrict the domain of the universal, we have $\forall(x, y)$ ($(x, y) \in Pos \rightarrow (x+y$ is an even integer)). Since the universal is true, ($(1, 0) \in Pos \rightarrow (1 + 0$ is an even integer)) is a true statement. Evaluating the conclusion of this conditional statement: By definition of long division, since $1 = 0 \cdot 2 + 1$ (where $0 \in \mathbb{Z}$ and $1 \in \mathbb{Z}$ and $0 \leq 1 < 2$ mean that 0 is the quotient and 1 is the remainder), 1 **mod** $2 = 1$ which is not 0 so the conclusion is false. A true conditional with a false conclusion must have a false hypothesis: $(1, 0) \notin Pos$, QED. $\square$

Proof of lemma by structural induction:

Basis Step:

Recursive Step: Consider arbitrary $(x, y) \in Pos$. To show is:

$(x + y$ is an even integer$) \rightarrow$ (sum of coordinates of next position is even integer)

Assume **as the induction hypothesis, IH** that:

CC BY-NC-SA 2.0 Version October 30, 2026 (7)

Week 6 Wednesday: Mathematical and Strong Induction

Proof by Mathematical Induction
To prove a universal quantification over the set of all integers greater than or equal to some base integer b,

Basis Step: Show the property holds for b.

Recursive Step: Consider an arbitrary integer n greater than or equal to b, assume (as the **induction hypothesis**) that the property holds for n, and use this and other facts to prove that the property holds for $n+1$.

The set $\mathbb{N}$ is recursively defined. Therefore, the function $sumPow : \mathbb{N} \to \mathbb{N}$ which computes, for input i, the sum of the nonnegative powers of 2 up to and including exponent i is defined recursively by

Basis step: $sumPow(0) = 1$
Recursive step: If $x \in \mathbb{N}$, then $sumPow(x+1) = sumPow(x) + 2^{x+1}$

$sumPow(0) =$

$sumPow(1) =$

$sumPow(2) =$

Fill in the blanks in the following proof of

$$\forall n \in \mathbb{N}\ (sumPow(n) = 2^{n+1} - 1)$$

Proof: Since $\mathbb{N}$ is recursively defined, we proceed by _____.

Basis case: We need to show that _____. Evaluating each side: $LHS = sumPow(0) = 1$ by the basis case in the recursive definition of $sumPow$; $RHS = 2^{0+1} - 1 = 2^1 - 1 = 2 - 1 = 1$. Since $1 = 1$, the equality holds.

Recursive case: Consider arbitrary natural number n and assume, as the _____ that $sumPow(n) = 2^{n+1} - 1$. We need to show that _____. Evaluating each side:

$$LHS = sumPow(n+1) \overset{\text{rec def}}{=} sumPow(n) + 2^{n+1} \overset{\text{IH}}{=} (2^{n+1} - 1) + 2^{n+1}.$$

$$RHS = 2^{(n+1)+1} - 1 \overset{\text{exponent rules}}{=} 2 \cdot 2^{n+1} - 1 = \left(2^{n+1} + 2^{n+1}\right) - 1 \overset{\text{regrouping}}{=} (2^{n+1} - 1) + 2^{n+1}$$

Thus, $LHS = RHS$. The structural induction is complete and we have proved the universal generalization. □

CC BY-NC-SA 2.0 Version October 30, 2026 (8)

Proof by Strong Induction

To prove that a universal quantification over the set of all integers greater than or equal to some base integer b holds, pick a fixed nonnegative integer j and then:

Basis Step:	Show the statement holds for b, $b+1$, …, $b+j$.
Recursive Step:	Consider an arbitrary integer n greater than or equal to $b+j$, assume (as the **strong induction hypothesis**) that the property holds for **each of** b, $b+1$, …, n, and use this and other facts to prove that the property holds for $n+1$.

Theorem: Every positive integer is a sum of (one or more) distinct powers of 2. *Binary expansions exist!*

The idea in the "Least significant first" algorithm for computing binary expansions is that the binary expansion of *half* a number becomes *part* of the binary expansion of the number of itself. We can use this idea in a proof by strong induction that binary expansions exist for all positive integers n.

Proof by strong induction, with $b = 1$ and $j = 0$. We need to prove that for each positive integer n, there is a positive integer k and coefficients $a_0, \ldots, a_{k-1}$ where each a_i is 0 or 1 and $a_{k-1} \neq 0$, and

$$n = \sum_{i=0}^{k-1} a_i 2^i$$

Basis step: WTS property is true about 1.

Recursive step: Consider an arbitrary integer $n \geq 1$.

Assume (as the strong induction hypothesis, IH) that the property is true about each of $1, \ldots, n$.

WTS that the property is true about $n+1$.

Idea: We will apply the IH to $(n+1)$ **div** 2.

Why is this ok?

CC BY-NC-SA 2.0 Version October 30, 2026 (9)

Why is this helpful?

By the IH, we can write $(n+1)$ **div** 2 as a sum of powers of 2. In other words, there are values $a_{k-1}, \ldots, a_0$ such that each a_i is 0 or 1, $a_{k-1} = 1$, and

$$\sum_{i=0}^{k-1} a_i 2^i = (n+1) \textbf{ div } 2$$

Define the collection of coefficients

$$c_j = \begin{cases} a_{j-1} & \text{if } 1 \leq j \leq k \\ (n+1) \textbf{ mod } 2 & \text{if } j = 0 \end{cases}$$

Calculating:

$$\begin{aligned}
\sum_{j=0}^{k} c_j 2^j &= c_0 + \sum_{j=1}^{k} c_j 2^j = c_0 + \sum_{i=0}^{k-1} c_{i+1} 2^{i+1} && \text{re-indexing the summation} \\
&= c_0 + 2 \cdot \sum_{i=0}^{k-1} c_{i+1} 2^i && \text{factoring out a 2 from each term in the sum} \\
&= c_0 + 2 \cdot \sum_{i=0}^{k-1} a_i 2^i && \text{by definition of } c_{i+1} \\
&= c_0 + 2\,(\ (n+1) \textbf{ div } 2\ ) && \text{by IH} \\
&= (\ (n+1) \textbf{ mod } 2\ ) + 2\,(\ (n+1) \textbf{ div } 2\ ) && \text{by definition of } c_0 \\
&= n+1 && \text{by definition of long division}
\end{aligned}$$

Thus, $n+1$ can be expressed as a sum of powers of 2, as required.

CC BY-NC-SA 2.0 Version October 30, 2026 (10)

Representing positive integers with primes

Theorem: Every positive integer *greater than 1* is a product of (one or more) primes.

Before we prove, let's try some examples:

$20 =$

$100 =$

$5 =$

Proof by strong induction, with $b = 2$ and $j = 0$.

Basis step: WTS property is true about 2.

Since 2 is itself prime, it is already written as a product of (one) prime.

Recursive step: Consider an arbitrary integer $n \geq 2$. Assume (as the strong induction hypothesis, IH) that the property is true about each of $2, \ldots, n$. WTS that the property is true about $n + 1$: We want to show that $n + 1$ can be written as a product of primes. Notice that $n + 1$ is itself prime or it is composite.

Case 1: assume $n + 1$ is prime and then immediately it is written as a product of (one) prime so we are done.

Case 2: assume that $n + 1$ is composite so there are integers x and y where $n + 1 = xy$ and each of them is between 2 and n (inclusive). Therefore, the induction hypothesis applies to each of x and y so each of these factors of $n + 1$ can be written as a product of primes. Multiplying these products together, we get a product of primes that gives $n + 1$, as required.

Since both cases give the necessary conclusion, the proof by cases for the recursive step is complete.

CC BY-NC-SA 2.0 Version October 30, 2026 (11)

Sending old-fashioned mail with postage stamps

Suppose we had postage stamps worth 5 cents and 3 cents. Which number of cents can we form using these stamps? In other words, which postage can we pay?

11?

15?

4?

$$CanPay(0) \wedge \neg CanPay(1) \wedge \neg CanPay(2) \wedge$$
$$CanPay(3) \wedge \neg CanPay(4) \wedge CanPay(5) \wedge CanPay(6)$$
$$\neg CanPay(7) \wedge \forall n \in \mathbb{Z}^{\geq 8} CanPay(n)$$

where the predicate $CanPay$ with domain $\mathbb{N}$ is

$$CanPay(n) = \exists x \in \mathbb{N} \exists y \in \mathbb{N}(5x + 3y = n)$$

Proof (idea): First, explicitly give witnesses or general arguments for postages between 0 and 7. To prove the universal claim, we can use mathematical induction or strong induction.

Approach 1, mathematical induction: if we have stamps that add up to n cents, need to use them (and others) to give $n + 1$ cents. How do we get 1 cent with just 3-cent and 5-cent stamps?
Either take away a 5-cent stamps and add two 3-cent stamps,
or take away three 3-cent stamps and add two 5-cent stamps.
The details of this proof by mathematical induction are making sure we have enough stamps to use one of these approaches.

Approach 2, strong induction: assuming we know how to make postage for **all** smaller values (greater than or equal to 8), when we need to make $n+1$ cents, add one 3 cent stamp to however we make $(n + 1) - 3$ cents.
The details of this proof by strong induction are making sure we stay in the domain of the universal when applying the induction hypothesis.

CC BY-NC-SA 2.0 Version October 30, 2026 (12)

Week 6 Friday Recursive Data Structures

Finding a winning strategy for a game

Consider the following game: two players start with two (equal) piles of jellybeans in front of them. They take turns removing any positive integer number of jellybeans at a time from one of two piles in front of them in turns.

The player who removes the last jellybean wins the game.

Which player (if any) has a strategy to guarantee to win the game?

Try out some games, starting with 1 jellybean in each pile, then 2 jellybeans in each pile, then 3 jellybeans in each pile. Who wins in each game?

Notice that reasoning about the strategy for the 1 jellybean game is easier than about the strategy for the 2 jellybean game.

Formulate a winning strategy by working to transform the game to a simpler one we know we can win.

CC BY-NC-SA 2.0 Version October 30, 2026 (13)

Player 2's Strategy: Take the same number of jellybeans that Player 1 did, but from the opposite pile.

Why is this a good idea: If Player 2 plays this strategy, at the next turn Player 1 faces a game with the same setup as the original, just with fewer jellybeans in the two piles. Then Player 2 can keep playing this strategy to win.

Claim: Player 2's strategy guarantees they will win the game.

Proof: By strong induction, we will prove that for all positive integers n, Player 2's strategy guarantees a win in the game that starts with n jellybeans in each pile.

Basis step: WTS Player 2's strategy guarantees a win when each pile starts with 1 jellybean.

In this case, Player 1 has to take the jellybean from one of the piles (because they can't take from both piles at once). Following the strategy, Player 2 takes the jellybean from the other pile, and wins because this is the last jellybean.

Recursive step: Let n be a positive integer. As the strong induction hypothesis, assume that Player 2's strategy guarantees a win in the games where there are $1, 2, \ldots, n$ many jellybeans in each pile at the start of the game.

WTS that Player 2's strategy guarantees a win in the game where there are $n+1$ jellybeans in each pile at the start of the game.

In this game, the first move has Player 1 take some number, call it c (where $1 \leq c \leq n+1$), of jellybeans from one of the piles. Playing according to their strategy, Player 2 then takes the same number of jellybeans from the other pile.

Notice that $(c = n+1) \vee (c \leq n)$.

Case 1: Assume $c = n+1$, then in their first move, Player 2 wins because they take all of the second pile, which includes the last jellybean.

Case 2: Assume $c \leq n$. Then after Player 2's first move, the two piles have an equal number of jellybeans. The number of jellybeans in each pile is

$$(n+1) - c$$

and, since $1 \leq c \leq n$, this number is between 1 and n. Thus, at this stage of the game, the game appears identical to a new game where the two piles have an equal number of jellybeans between 1 and n. Thus, the strong induction hypothesis applies, and Player 2's strategy guarantees they win.

CC BY-NC-SA 2.0 Version October 30, 2026 (14)

Definition The set of linked lists of natural numbers L is defined recursively by

Basis Step:	$[] \in L$
Recursive Step:	If $l \in L$ and $n \in \mathbb{N}$, then $(n, l) \in L$

Visually:

Example: the list with two nodes whose first node has 20 and whose second node has 42

Definition: The length of a linked list of natural numbers L, $length : L \to \mathbb{N}$ is defined by

Basis Step:		$length(\ [] \ )$	$= 0$
Recursive Step:	If $l \in L$ and $n \in \mathbb{N}$, then	$length(\ (n, l)\ )$	$= 1 + length(l)$

Definition: The function $prepend : L \times \mathbb{N} \to L$ that adds an element at the front of a linked list is defined by

Definition The function $append : L \times \mathbb{N} \to L$ that adds an element at the end of a linked list is defined by

Basis Step:	If $m \in \mathbb{N}$ then
Recursive Step:	If $l \in L$ and $n \in \mathbb{N}$ and $m \in \mathbb{N}$, then

CC BY-NC-SA 2.0 Version October 30, 2026 (15)

Claim: $\forall l \in L$ ($length($ $append($ $(l, 100)$ $)$ $) > length(l)$)

Proof: By structural induction on L, we have two cases:

Basis Step

1. **To Show** $length($ $append($ $([], 100)$ $)$ $) > length($ [] $)$	Because [] is the only element defined in the basis step of L, we only need to prove that the property holds for [].
2. **To Show** $length($ $(100, [])$ $) > length($ [] $)$	By basis step in definition of *append*.
3. **To Show** $(1 + length($ [] $))$ $> length($ [] $)$	By recursive step in definition of *length*.
4. **To Show** $1 + 0 > 0$	By basis step in definition of *length*.
5. T	By properties of integers
QED	Because we got to T only by rewriting **To Show** to equivalent statements, using well-defined proof techniques, and applying definitions.

Recursive Step

Consider an arbitrary: $l' \in L$, $n \in \mathbb{N}$, and we assume as the **induction hypothesis** that:

$$length($ $append($ $(l', 100$ $)$ $) > length(l')$$

Our goal is to show that $length($ $append($ $($ $(n, l'), 100$ $)$ $) > length($ (n, l')) is also true. We start by working with one side of the candidate inequality:

$$\begin{aligned}
LHS &= length(\ append(\ (\ (n, l'), 100\ )\ )\ ) \\
&= length(\ (n, append(\ (l', 100)\ )\ )\ ) && \text{by the recursive definition of } append \\
&= 1 + length(\ append(\ (l', 100)\ )\ ) && \text{by the recursive definition of } length \\
&> 1 + length(l') && \text{by the induction hypothesis} \\
&= length((n, l')) && \text{by the recursive definition of } length \\
&= RHS
\end{aligned}$$

CC BY-NC-SA 2.0 Version October 30, 2026 (16)

Prove or disprove: $\forall n \in \mathbb{N}\ \exists l \in L\ (\ length(l) = n\ )$

CC BY-NC-SA 2.0 Version October 30, 2026 (17)